

Análise Multi-Arquitetural e Métodos de Otimização no Produto Matricial Denso em Sistemas Modernos

1. Introdução e Fundamentação Teórica

A operação de multiplicação de matrizes densas é um dos pilares centrais da computação científica de alto desempenho (HPC), servindo como primitivo computacional fundamental em simulações numéricas, algoritmos de renderização gráfica, processamento de sinais e, contemporaneamente, na infraestrutura de treinamento e inferência de Redes Neurais Profundas (DNNs). Formalmente, para duas matrizes reais A de dimensão $M \times K$ e B de dimensão $K \times N$, o produto matricial $C = A \times B$ gera uma matriz resultante C de dimensão $M \times N$, cujos elementos são definidos por:

$$C_{i,j} = \sum_{k=0}^{K-1} A_{i,k} \cdot B_{k,j}$$

Do ponto de vista da teoria da complexidade computacional, o algoritmo ingênuo estruturado em laços triplamente aninhados exige exatos $M \cdot N \cdot K$ multiplicações e $M \cdot N \cdot (K - 1)$ adições. Para o caso de matrizes quadradas de ordem N , o algoritmo requer $2N^3 - N^2$ operações de ponto flutuante, estabelecendo uma complexidade temporal assintótica de $O(N^3)$ e espacial de $O(N^2)$ para o armazenamento contíguo dos dados em memória secundária ou primária.

Entretanto, a complexidade assintótica teórica falha em prever o desempenho prático de rotinas numéricas em arquiteturas superescalares modernas. O fator determinante para o rendimento computacional é o fenômeno do *Memory Wall* — a lacuna crescente entre a taxa de throughput aritmético das Unidades de Ponto Flutuante (FPUs) e a latência/largura de banda da memória principal (DRAM). Sem otimizações focadas no reaproveitamento de dados na hierarquia de cache, a CPU despende a maior parte dos ciclos de clock paralisada em estados de espera (*stalls*), caracterizando um regime estritamente limitado pela memória (*Memory-Bound*).

2. Engenharia de Software e Princípios de Otimização Algorítmica

A implementação desenvolvida no módulo `matmul.c` substitui a abordagem ingênua por técnicas avançadas de otimização no nível de linguagem de sistemas e compilador:

- Alinhamento Estrito de Memória (64 Bytes): Utilização da rotina POSIX `posix_memalign()` para garantir que os endereços de memória base dos ponteiros das matrizes sejam múltiplos exatos de 64 bytes. Em arquiteturas x86-64 modernas, as linhas de cache possuem 64 bytes. O alinhamento previne a ocorrência de acessos divididos entre duas linhas de cache (*cache line splits*) e assegura que as instruções de carregamento vetorial SIMD (como AVX2 e AVX-512) operem com latência mínima.

- Reordenação de Laços (i-k-j) para Localidade Espacial: A substituição do padrão tradicional (i-j-k) pela permutação (i-k-j) garante acessos estritamente sequenciais à matriz B e à matriz C no laço mais interno. Como a linguagem C armazena matrizes em ordem de linha principal (*row-major order*), o acesso contíguo permite ao *Hardware Prefetcher* identificar o padrão de varredura e pré-carregar as linhas de cache subsequentes antes da requisição explícita da CPU.
- Eliminação de Aliasing com Qualificadores Restritos: A inclusão do qualificador `restrict` informa ao compilador que os ponteiros `ptr_A`, `ptr_B` e `ptr_C` não compartilham o mesmo espaço de endereçamento. Somado às rotinas intrínsecas `__builtin_assume_aligned()`, elimina-se a necessidade de checagens defensivas em tempo de execução, liberando a vetorização automática (*auto-vectorization*).

A métrica de medição isolou o tempo de computação pura (`TEMPO_COMP`) utilizando a chamada de sistema `clock_gettime(CLOCK_MONOTONIC)`, garantindo precisão na ordem dos nanosegundos sem interferência de variações do relógio do sistema. A vazão real foi aferida em GFLOPS:

$$\text{GFLOPS} = (2 \times M \times K \times N) / (\text{TEMPO_COMP} \times 10^9)$$

O fator multiplicativo decorre do fato de a iteração mais interna exigir estritamente uma multiplicação e uma adição acumulada. O custo algorítmico cresce cubicamente: uma matriz de 1.000×1.000 requer 2 bilhões de operações; contudo, a matriz de 4.000×4.000 exige 128 bilhões de operações de ponto flutuante, originando um cenário de esforço extremo para a largura de banda da memória e para as caches.

3. Profiling Arquitetural e Análise de Entrada/Saída (E/S)

A avaliação experimental considerou heterogeneidade microarquitetural de três processadores distintos, representativos de diferentes gerações de arquitetura x86-64:

Integrante	CPU / Microarquitetura	Frequência (Min / Máx / BogoMIPS)	Hierarquia de Cache (L1d / L2 / L3)	Extensões Relevantes
Roque	Intel Core i7-13620H (13ª Ger.)	400 MHz / 4900 MHz	416 KiB / 9,5 MiB / 24 MiB	AVX2, FMA, SSSE3

Felipe	AMD Ryzen 5 8600G (Zen 4)	N/D (BogoMIPS: 8699.51)	192 KiB / 6 MiB / 16 MiB	AVX2, AVX-512F, FMA
Eduardo	AMD Ryzen 5 3500U (Zen+)	N/D (BogoMIPS: 4199.87)	128 KiB / 2 MiB / 4 MiB	AVX2, FMA, SSSE3

A análise das operações de E/S revelou que rotinas orientadas a texto (arquivos CSV) introduzem um gargalo léxico e de conversão de tipos (ASCII/UTF-8 para ponto flutuante IEEE 754 de 32 bits) extremamente custoso, dominando o tempo total de execução. A substituição por arquivos binários planos permitiu o mapeamento direto para a RAM via chamadas `fread()`, reduzindo a latência de carregamento a valores insignificantes:

O encapsulamento no padrão proprietário binário elimina essa latência. Os bytes, previamente formatados na disposição nativa *Little-Endian*, são mapeados diretamente via `fread`.

Ordem da Matriz (N×N)	E/S Texto (.csv)	E/S Binário (.bin) (s)
1.000 × 1.000	N/D (Gargalo Léxico)	0.049790
2.000 × 2.000	N/D (Gargalo Léxico)	0.187480
4.000 × 4.000	N/D (Gargalo Léxico)	0.731171

A transição elimina o mapeamento indireto de texto, efetuando a transferência direta de dados do SSD para as posições alocadas na memória RAM através de blocos contíguos.

4. Análise Empírica e Fenomenologia de Caches

A tabela a seguir consolida o desempenho obtido em GFLOPS nas três arquiteturas sob diferentes sinalizadores de compilação GCC extraídos empiricamente:

Dimensão (N)	Flags Compilador	i7-13620H (Roque)	R5 8600G (Felipe)	R5 3500U (Eduardo)
1000 × 1000	-O0 (Sem otimização)	0.27 GFLOPS	1.05 GFLOPS	1.05 GFLOPS
1000 × 1000	-O3 -march=native -ffast-math	53.52 GFLOPS	50.52 GFLOPS	19.87 GFLOPS
2000 × 2000	-O3 -march=native -ffast-math	56.67 GFLOPS	36.49 GFLOPS	19.46 GFLOPS
4000 × 4000	-O3 -march=native -ffast-math	56.40 GFLOPS	17.55 GFLOPS	18.06 GFLOPS

O Cenário Escalar e as Flags Associativas: No nível nulo de otimização (-O0), focado apenas em depuração, não existe alocação de variáveis em registradores operacionais. Todas as operações aritméticas são espelhadas compulsoriamente na pilha de memória (*stack*). A latência é provocada pelas movimentações intermitentes ininterruptas: o processador move dados da RAM para a cache, da cache para o registrador, invoca a adição ou multiplicação e devolve à RAM. Isso converte a matriz em um problema limitado pela memória (*Memory-Bound*), gargalando o motor aritmético em pouco mais de 1 GFLOPS ou menos em todas as máquinas.

A elevação exponencial de desempenho observada ao ativar -ffast-math decorre da relaxação das regras estritas do padrão IEEE 754 no tocante à não-associatividade do ponto flutuante. O compilador assume permissão para

reagrupar as adições acumuladas em múltiplos registradores vetoriais simultâneos, reduzindo dependências de dados entre instruções consecutivas.

Resiliência Arquitetural e o Cache Thrashing: A discrepância mais notável deste projeto revela-se ao observarmos o comportamento de cada chip face ao esgotamento das estruturas de cache em matrizes de alta ordem. Para $N=4000$ (precisão simples de 32 bits), o volume total consumido pelas três matrizes é calculado por:

$$\text{Memória Total} = 3 \times (4000 \times 4000 \times 4 \text{ bytes}) = 192 \text{ MB}$$

Como nenhuma das arquiteturas analisadas possui cache L3 com capacidade para comportar 192 MB de dados contíguos (sendo 16 MB no Zen 4 de Felipe, 24 MB no Alder Lake de Roque e 4 MB no Zen+ de Eduardo), a integridade temporal da cache é violada, propiciando o cenário de *Cache Thrashing*.

Contudo, os resultados práticos expuseram respostas microarquiteturais dramaticamente diferentes a este gargalo:

- Na máquina de Felipe (Zen 4), a vazão degradou de forma severa, despencando de seu pico de 50.52 GFLOPS (onde a matriz de 1000 cabia na L3) para apenas 17.55 GFLOPS em $N=4000$. O barramento de memória gargalou a FPU.
- Em contrapartida, a máquina de Roque (Alder Lake) demonstrou uma estabilidade surpreendente, processando 53.52 GFLOPS no início e mantendo 56.40 GFLOPS no cenário extremo de $N=4000$. A despeito do esgotamento da cache L3 de 24 MB, o *Hardware Prefetcher* inteligente do Intel i7 e a provável enorme largura de banda de memória (DDR5) do sistema mascararam efetivamente a latência da DRAM, impedindo a CPU de sofrer *stalls* significativos.
- Na máquina de Eduardo (Zen+), as limitações de instruções (ausência de expansões hiper-largas como AVX-512) estabilizaram o sistema em um teto em torno de 19.87 GFLOPS, degradando apenas ligeiramente em matrizes maiores devido à incapacidade da ALU legada de exigir mais dados do barramento do que o limite estipulado.

5. Dissecação e Análise do Código de Montagem (Assembly x86-64)

A validação definitiva da restrição de ponteiros e da otimização reside nos artefatos em código de montagem (*Assembly*) produzidos nos ambientes locais. A Abordagem Escalar (Intel Alder Lake de Roque sob -O0):

Snippet de código

```
# src/matmul.c:60: ptr_C[i * N + j] += a_ik * ptr_B[k * N + j];  
    movss  xmm0, DWORD PTR [rax]  
    mulss  xmm0, DWORD PTR -204[rbp]
```

```
addss  xmm0, xmm1
movss  DWORD PTR [rax], xmm0
```

A análise do código não otimizado gerado sob a flag `-O0` revela o uso estritamente escalar de registradores de precisão simples `xmm0` através das instruções `movss`, `mulss` e `addss`. Cada operação aritmética processa exatamente um escalar por ciclo, acompanhada por leituras e escritas redundantes no quadro de pilha (*stack frame*) apontado pelo registrador `rbp`.

Mutação Vetorial e Fusão de Instruções AVX (Arquiteturas sob `-O3 -march=native`):

Ao aplicar-se a flag nativa em processadores de última geração, o cenário transmuta-se completamente:

Snippet de código

```
# src/matmul.c:60: ptr_C[i * N + j] += a_ik * ptr_B[k * N + j];
    vmovups zmm6, ZMMWORD PTR [rdx+rdi*1]
    vfmadd213ps zmm6, zmm2, ZMMWORD PTR [rax+rdi*1]
    vmovups ZMMWORD PTR [rax+rdi*1], zmm6
```

A vetorização nativa reestrutura o laço interno aproveitando extensões vetoriais SIMD. No processador AMD Ryzen 8600G (arquitetura Zen 4 com suporte nativo a AVX-512), o compilador emprega os registradores vetoriais de 512 bits `zmm` juntamente com a instrução de fusão aritmética `vfmadd213ps` (*Fused Multiply-Add Single-Precision*). Esta instrução executa a operação de produto-acumulação $a \times b + c$ em 16 elementos de ponto flutuante de precisão simples simultaneamente em um único ciclo de clock, multiplicando em 16x a densidade operacional em comparação à abordagem escalar.

Essa fusão não apenas duplica o limite teórico de GFLOPS, como também adia o truncamento exigido pela norma IEEE 754 até que toda a etapa associativa interna seja concluída, reduzindo substancialmente a propagação de erros de arredondamento. Fica provado empírica e analiticamente que as limitações do paradigma legado são superadas pelo projeto consciente da localidade espacial e pelo aproveitamento vetorial do hardware subjacente.

6. Conclusões e Trabalhos Futuros

A investigação técnica comprova que o alinhamento de memória, a eliminação de aliasing, a reordenação de laços e o uso de flags avançadas de compilação (`-O3`, `-march=native`, `-ffast-math`) elevam o rendimento computacional do produto matricial em até dois ordens de grandeza. Contudo, para matrizes cujas dimensões excedem os limites da hierarquia de cache L3 ($N > 2000$), rotinas que dependem unicamente da capacidade do pré-buscador da CPU podem falhar

gravemente caso a largura de banda de memória seja subótima (como observado na arquitetura Zen 4).

Para mitigar esse limite físico em pesquisas futuras e nivelar a resiliência de cache em processadores vulneráveis ao *Cache Thrashing*, recomenda-se a implementação do algoritmo de bloqueio por blocos de cache (*Cache Tiling/Blocking*), aliado ao paralelismo multi-thread via OpenMP ou SIMD manual com intrínsecos AVX-512.